

End Term Project

Comprehensive SRE Implementation for a Distributed Microservices System

1. Title

End-to-End Implementation of Site Reliability Engineering Practices in a Multi-Orchestrated Microservices Infrastructure Using Docker Swarm, Kubernetes, Terraform, and Ansible

2. Abstract

This project presents a comprehensive implementation of Site Reliability Engineering (SRE) principles applied to a distributed microservices-based system. The system integrates containerization, multi-platform orchestration, monitoring, infrastructure provisioning, configuration management, incident response, and capacity planning.

The architecture consists of more than five independent microservices deployed using both Docker Swarm and Kubernetes to demonstrate comparative orchestration strategies. Infrastructure provisioning is performed using Terraform, while system configuration and deployment automation are managed using Ansible.

Observability is achieved through Prometheus and Grafana, enabling real-time monitoring and alerting. A simulated incident is introduced to evaluate system resilience, followed by a structured postmortem analysis. Additionally, automation mechanisms and capacity planning strategies are implemented to enhance reliability and scalability.

This project demonstrates a full SRE lifecycle, including system setup, monitoring, incident management, automation, and performance optimization.

3. Objectives

The primary objective of this project is to demonstrate the integration of SRE practices across the entire system lifecycle.

Specific objectives include:

1. Designing and deploying a distributed microservices architecture (6+ services)
2. Implementing multi-orchestration using Docker Swarm and Kubernetes
3. Provisioning infrastructure using Terraform
4. Automating configuration and deployment using Ansible

5. Defining SLIs and SLOs
6. Implementing monitoring and alerting systems
7. Simulating and resolving production incidents
8. Conducting postmortem analysis
9. Implementing automation and capacity planning strategies

4. System Overview

The system is a scalable web-based microservices application designed for SRE demonstration.

Microservices (≥ 6) **you can use any other services** for example:

1. Authentication Service — user login and security
2. Product Service — product catalog management
3. Order Service — order processing
4. Payment Service — payment handling simulation
5. Notification Service — email/alert simulation
6. User Profile Service — user data management

Supporting Components

1. Frontend: Nginx-based web interface
2. Database: PostgreSQL
3. Optional Message Broker: Redis / RabbitMQ
4. Monitoring: Prometheus + Grafana

5. Assignment Integration

Assignment 1 — Environment Setup

1. Docker environment setup
2. Containerized services
3. Docker Compose orchestration

Assignment 2 — SLI/SLO Design

SLIs:

1. Availability
2. Latency
3. Error rate
4. Request success rate

SLOs:

1. Availability $\geq 99\%$
2. Latency ≤ 200 ms
3. Error rate $\leq 1\%$

Assignment 3 — Monitoring

1. Prometheus metrics collection
2. Grafana dashboards
3. Alert configuration

Midterm Project

1. Initial microservices implementation
2. Functional system deployment

Assignment 4 — Incident Response

1. Simulated Order Service failure
2. Root cause analysis
3. Recovery process

Assignment 5 — Infrastructure as Code

1. Terraform-based provisioning
2. Automated VM creation

Assignment 6 — Automation & Capacity Planning

1. Automated deployment
2. Health checks and restart policies
3. Load analysis and scaling strategies

6. Multi-Orchestration Architecture

6.1 Docker Swarm

Used for:

1. Simple clustering
2. Service replication
3. Fast deployment

`docker swarm init`

`docker stack deploy -c docker-compose.yml app`

6.2 Kubernetes

Used for:

1. Advanced orchestration
2. Auto-scaling
3. Self-healing
4. Declarative deployments

Components:

1. Pods
2. Deployments
3. Services
4. ConfigMaps

6.3 Justification

Using both platforms enables:

1. Comparative analysis of orchestration tools
2. Demonstration of flexibility in deployment
3. Deeper understanding of SRE infrastructure design

7. Infrastructure Provisioning (Terraform)

Terraform is used to:

1. Provision cloud infrastructure (VMs)
2. Configure networking and access rules
3. Ensure reproducibility

Key Benefits

1. Declarative configuration
2. Version control
3. Automated provisioning

8. Configuration Management (Ansible)

Ansible automates:

1. Installation of Docker and Kubernetes
2. Node configuration
3. Deployment of services
4. Monitoring setup

Tasks

1. Install dependencies
2. Configure environment
3. Deploy application stack
4. Configure monitoring tools

9. System Architecture

User

|

Frontend (Nginx)

|

API Gateway

|

+-----+

| Microservices |

|-----|

| Auth | Product | Order | Payment |

| Notification | User Profile |

+-----+

|

PostgreSQL / Redis

Monitoring:

Prometheus → Grafana

Infrastructure:

Terraform → VM provisioning

Configuration:

Ansible → Setup & deployment

Orchestration:

Docker Swarm + Kubernetes

10. Monitoring and Observability

Tools

- Prometheus — metrics collection
- Grafana — visualization

Metrics

1. CPU usage
2. Memory usage
3. Request rate
4. Error rate
5. Service uptime

11. Incident Simulation

Scenario

Order Service fails due to incorrect database configuration.

Impact

1. Order creation unavailable
2. Partial service degradation

Response

1. Detection via monitoring
2. Log analysis
3. Configuration fix
4. Service restart

Outcome

1. System restored
2. Metrics normalized

12. Automation

1. Docker-based automated deployment
2. Ansible automation scripts
3. Service restart policies

4. Health checks
5. Monitoring alerts

13. Capacity Planning

Findings

1. Order and Payment services consume most resources
2. Database is a bottleneck

Strategies

1. Horizontal scaling (replicas)
2. Vertical scaling (CPU/RAM)
3. Database optimization

14. Results

The system demonstrates:

1. Multi-orchestrated deployment
2. Infrastructure automation
3. Reliable monitoring and alerting
4. Efficient incident handling
5. Scalable architecture

15. Conclusion

This project demonstrates a full implementation of SRE practices in a distributed system. By integrating multi-orchestration, infrastructure automation, monitoring, and incident response, the system achieves high reliability, scalability, and maintainability.

16. Deliverables

1. Microservices source code (6+ services)
2. Docker Compose / Swarm configuration
3. Kubernetes manifests
4. Terraform files
5. Ansible playbooks
6. Monitoring setup
7. Incident report and postmortem
8. Screenshots and demo evidence

9. Upload ONLY PDF WITH GIT LINK!!!